Transport-triggered processors

Jani Boutellier
Computer Science and Engineering Laboratory

This work is licensed under a Creative Commons Attribution 3.0 Unported License:
http://creativecommons.org/licenses/by/3.0/



What you will learn today

- What components does a TTA processor constitute of
- What TTA programs look like in machine code
- Basic optimization of TTA programs



Transport-triggered architecture

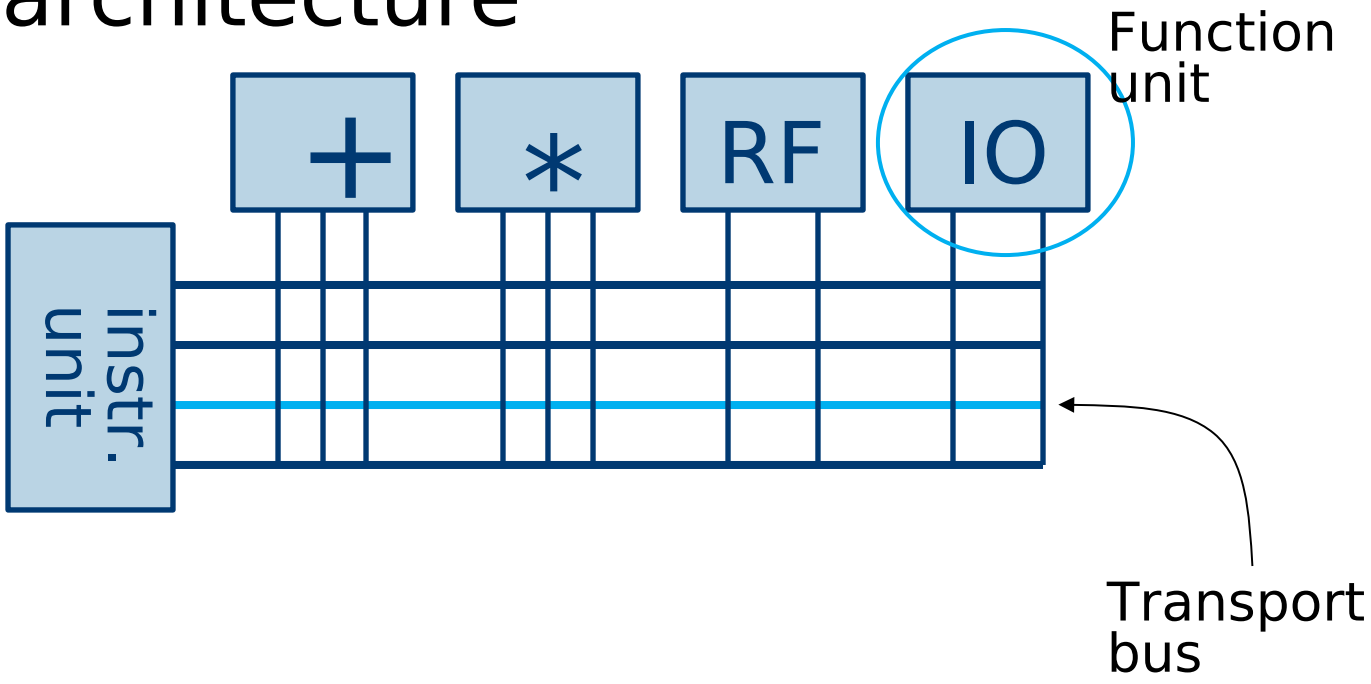
Transport-triggered architecture (TTA) processors

- An evolution of the VLIW
- Only 1 instruction: move data

  →
- Compiler needs to do a lot of work
- Can be very efficient
- Easy to design, scalable



Transport-triggered architecture



Transport-triggered architecture

- TTAs do not have an instruction set, instead, the programmer (compiler) directly defines data transports between functional units
- RISC, CISC and VLIW processor move data between FUs through registers. A TTA can directly send data from one FU to another – possibility to save power



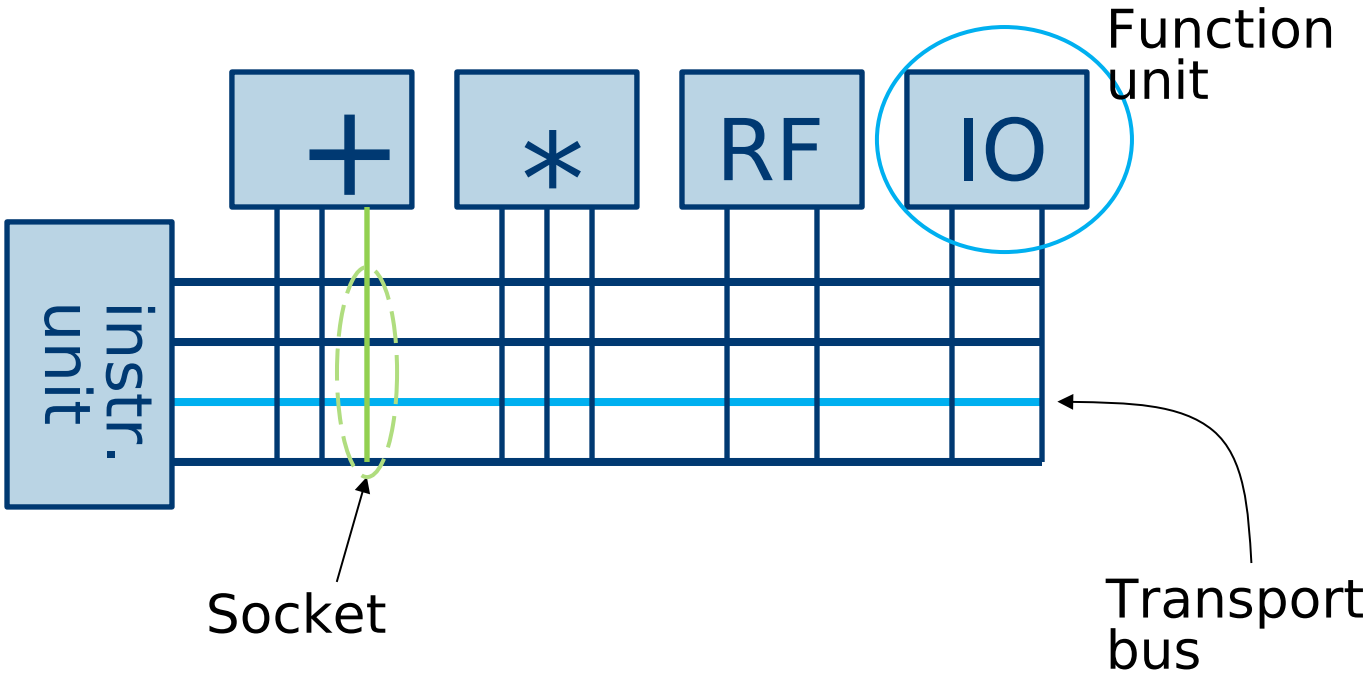
Transport-triggered architecture

- The general architecture of a TTA processor is very scalable: adding a new functional unit increases the complexity linearly
- The VLIW problem that TTA does not directly solve, is that of code density

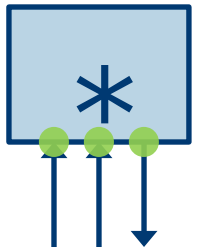
TTA structure



TTA processors

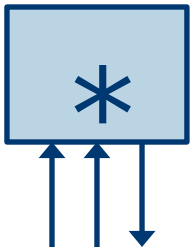


TTA processors



- Function units connect to sockets through ports

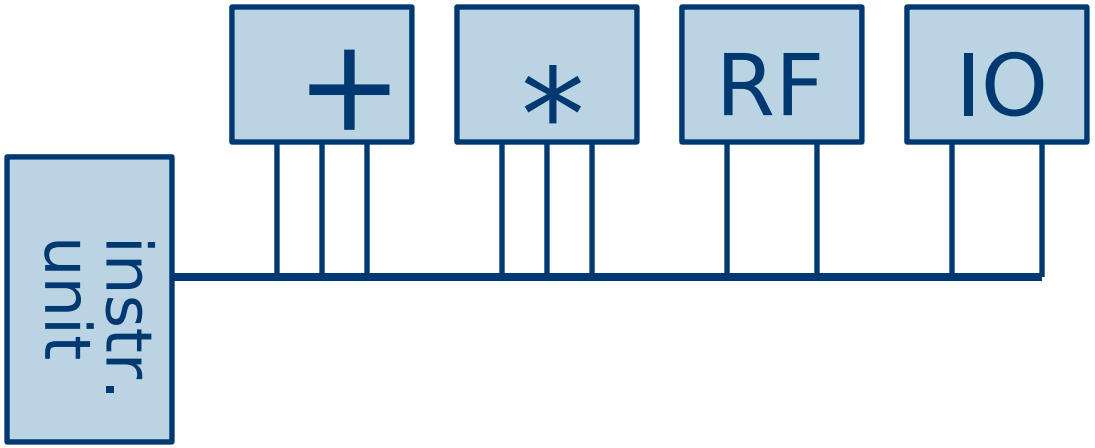
TTA processors



- Function units connect to sockets through ports
- Ports have either input or output direction
- This multiplier has two inputs for operands and one output for the result
- One of the inputs always *triggers* the FU

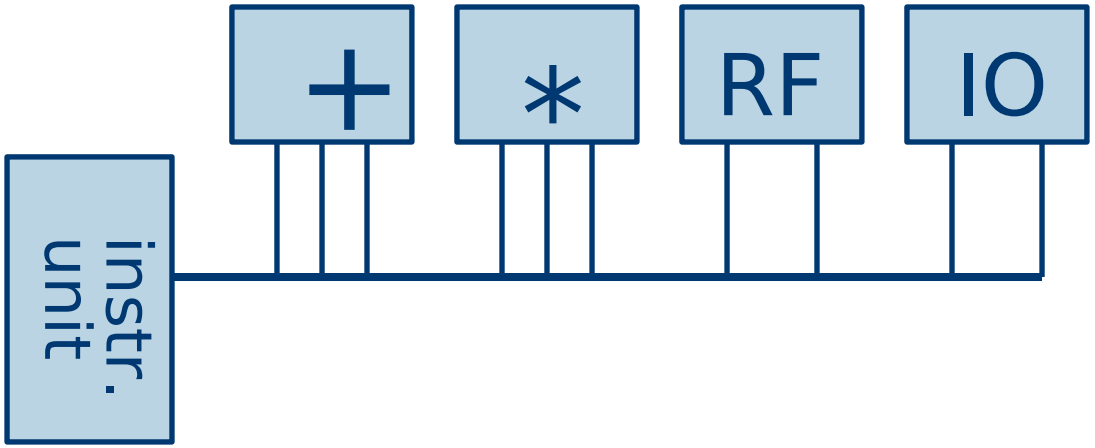
Computation example

Computation example




```
a = READ_IO();
b = READ_IO();
c = a + b * b;
WRITE_IO(c);
```

Computation example

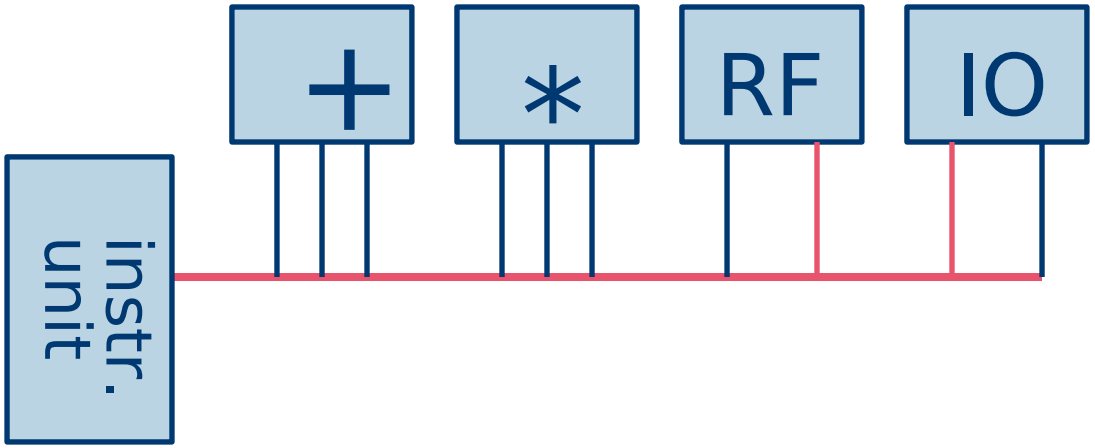



```
a = READ_IO();
b = READ_IO();
c = a + b * b;
WRITE_IO(c);
```

```
mov IO(0) -> RF(a0)     ; IO(0) is used to read data from outside
mov IO(0) -> RF(a1)     ; RF is a register file, to store data
mov RF(a1) -> mul(0)    ; mul(0) stores operand 1 of the multiplier
mov RF(a1) -> mul(1)    ; mul(1) stores operand 2 and triggers
mov mul(2) -> RF(a2)    ; mul(2) provides the multiplication result
mov RF(a0) -> add(0)    ; add(0) stores operand 1 of the adder
mov RF(a2) -> add(1)    ; b*b was stored to RF(a2) two lines before
mov add(2) -> IO(1)     ; IO(2) writes data to the outside
```




Computation example

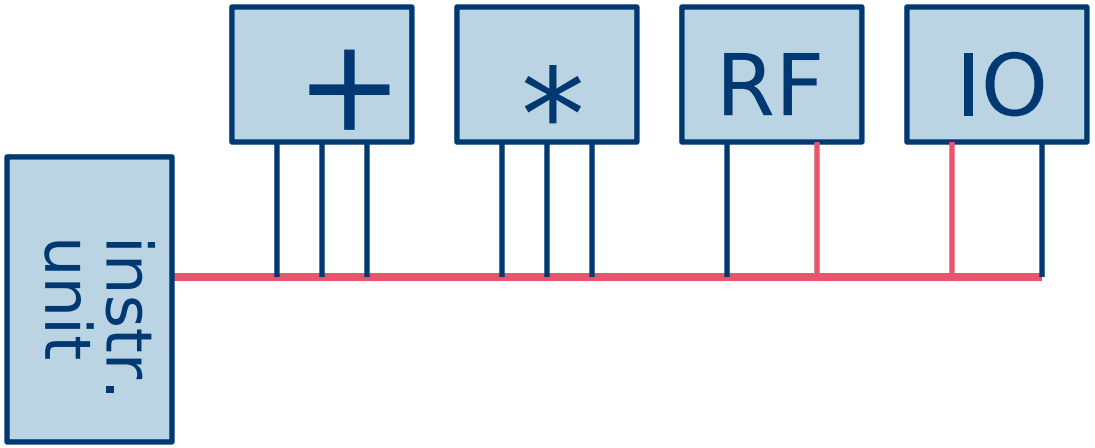



```
a = READ_IO();
b = READ_IO();
c = a + b * b;
WRITE_IO(c);
```

```
mov IO(0) -> RF(a0)      ; IO(0) is used to read data from outside
mov IO(0) -> RF(a1)      ; RF is a register file, to store data
mov RF(a1) -> mul(0)     ; mul(0) stores operand 1 of the multiplier
mov RF(a1) -> mul(1)     ; mul(1) stores operand 2 and triggers
mov mul(2) -> RF(a2)     ; mul(2) provides the multiplication result
mov RF(a0) -> add(0)     ; add(0) stores operand 1 of the adder
mov RF(a2) -> add(1)     ; b*b was stored to RF(a2) two lines before
mov add(2) -> IO(1)      ; IO(2) writes data to the outside
```




Computation example

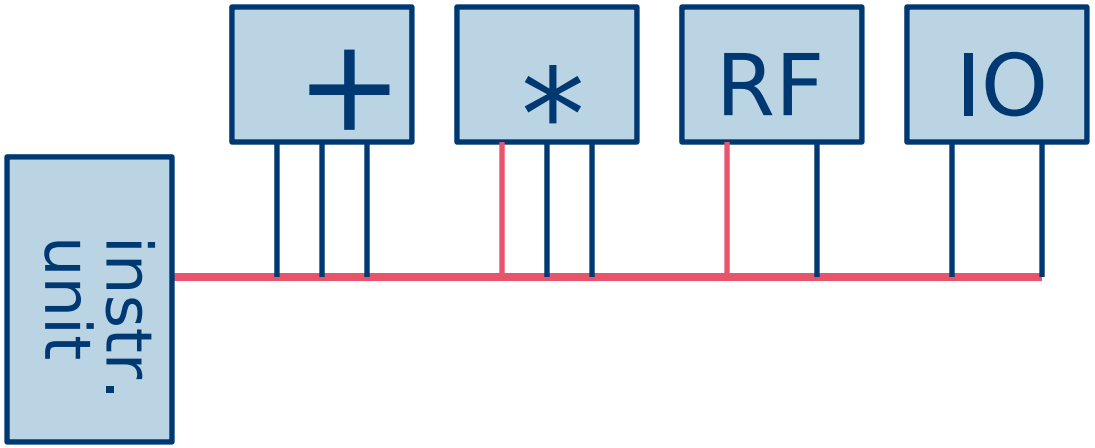



```
a = READ_IO();
b = READ_IO();
c = a + b * b;
WRITE_IO(c);
```

```
mov IO(0) -> RF(a0)     ; IO(0) is used to read data from outside
mov IO(0) -> RF(a1)     ; RF is a register file, to store data
mov RF(a1) -> mul(0)    ; mul(0) stores operand 1 of the multiplier
mov RF(a1) -> mul(1)    ; mul(1) stores operand 2 and triggers
mov mul(2) -> RF(a2)    ; mul(2) provides the multiplication result
mov RF(a0) -> add(0)    ; add(0) stores operand 1 of the adder
mov RF(a2) -> add(1)    ; b*b was stored to RF(a2) two lines before
mov add(2) -> IO(1)     ; IO(2) writes data to the outside
```




Computation example

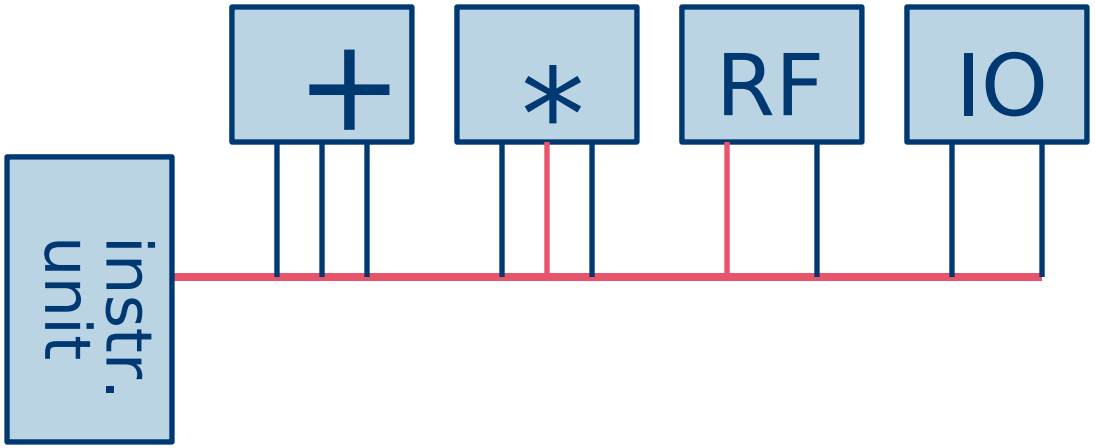



```
a = READ_IO();
b = READ_IO();
c = a + b * b;
WRITE_IO(c);
```

```
mov IO(0) -> RF(a0)     ; IO(0) is used to read data from outside
mov IO(0) -> RF(a1)     ; RF is a register file, to store data
mov RF(a1) -> mul(0)    ; mul(0) stores operand 1 of the multiplier
mov RF(a1) -> mul(1)    ; mul(1) stores operand 2 and triggers
mov mul(2) -> RF(a2)    ; mul(2) provides the multiplication result
mov RF(a0) -> add(0)    ; add(0) stores operand 1 of the adder
mov RF(a2) -> add(1)    ; b*b was stored to RF(a2) two lines before
mov add(2) -> IO(1)     ; IO(2) writes data to the outside
```




Computation example

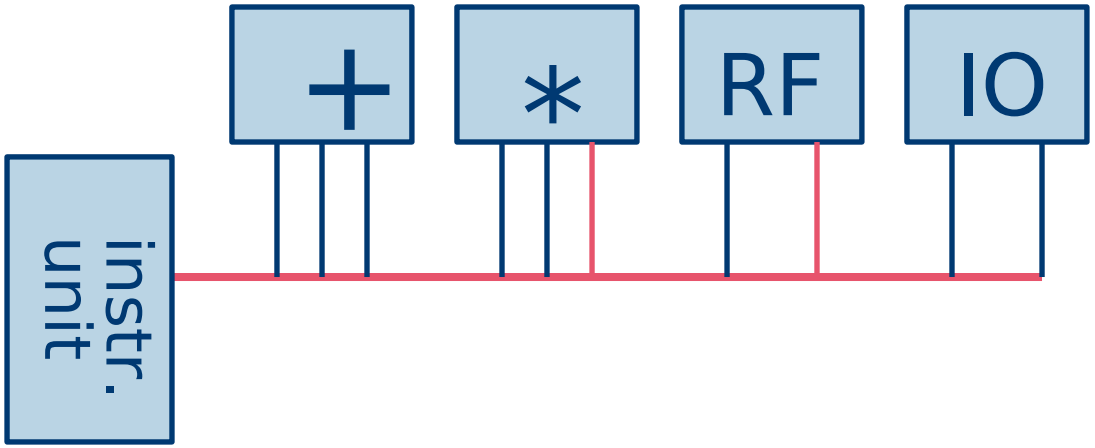



```
a = READ_IO();
b = READ_IO();
c = a + b * b;
WRITE_IO(c);
```

```
mov IO(0) -> RF(a0)     ; IO(0) is used to read data from outside
mov IO(0) -> RF(a1)     ; RF is a register file, to store data
mov RF(a1) -> mul(0)    ; mul(0) stores operand 1 of the multiplier
mov RF(a1) -> mul(1)    ; mul(1) stores operand 2 and triggers
mov mul(2) -> RF(a2)    ; mul(2) provides the multiplication result
mov RF(a0) -> add(0)    ; add(0) stores operand 1 of the adder
mov RF(a2) -> add(1)    ; b*b was stored to RF(a2) two lines before
mov add(2) -> IO(1)     ; IO(2) writes data to the outside
```




Computation example

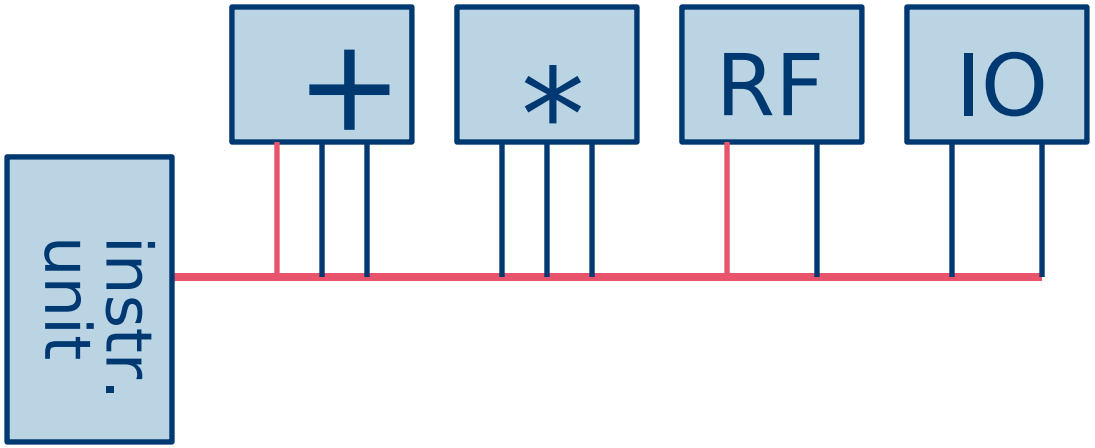



```
a = READ_IO();
b = READ_IO();
c = a + b * b;
WRITE_IO(c);
```

```
mov IO(0) -> RF(a0)     ; IO(0) is used to read data from outside
mov IO(0) -> RF(a1)     ; RF is a register file, to store data
mov RF(a1) -> mul(0)    ; mul(0) stores operand 1 of the multiplier
mov RF(a1) -> mul(1)    ; mul(1) stores operand 2 and triggers
mov mul(2) -> RF(a2)    ; mul(2) provides the multiplication result
mov RF(a0) -> add(0)    ; add(0) stores operand 1 of the adder
mov RF(a2) -> add(1)    ; b*b was stored to RF(a2) two lines before
mov add(2) -> IO(1)     ; IO(2) writes data to the outside
```



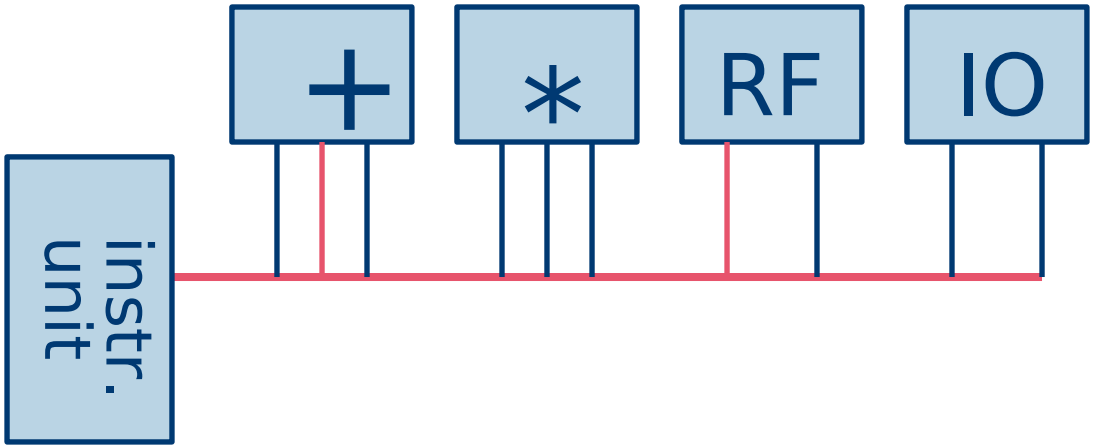

Computation example




```
a = READ_IO();
b = READ_IO();
c = a + b * b;
WRITE_IO(c);
```

```
mov IO(0) -> RF(a0)     ; IO(0) is used to read data from outside
mov IO(0) -> RF(a1)     ; RF is a register file, to store data
mov RF(a1) -> mul(0)    ; mul(0) stores operand 1 of the multiplier
mov RF(a1) -> mul(1)    ; mul(1) stores operand 2 and triggers
mov mul(2) -> RF(a2)    ; mul(2) provides the multiplication result
mov RF(a0) -> add(0)    ; add(0) stores operand 1 of the adder
mov RF(a2) -> add(1)    ; b*b was stored to RF(a2) two lines before
mov add(2) -> IO(1)     ; IO(2) writes data to the outside
```

Computation example

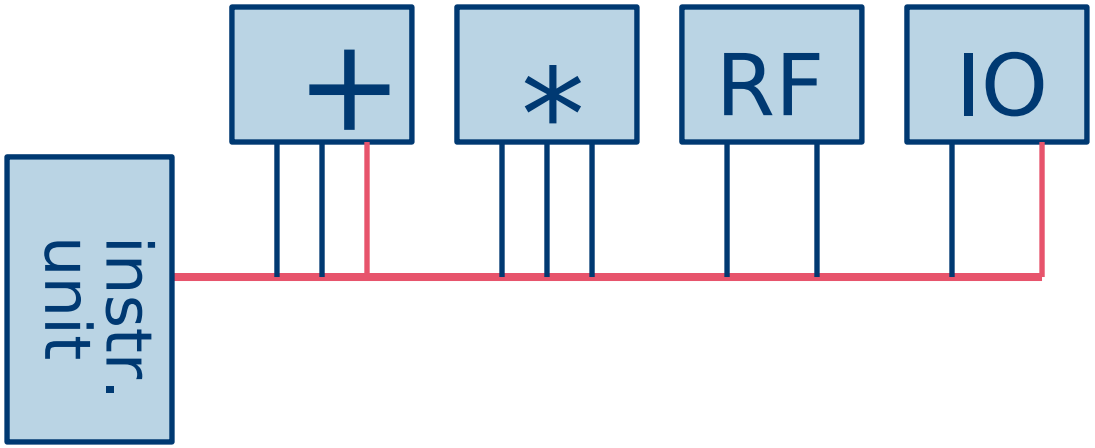



```
a = READ_IO();
b = READ_IO();
c = a + b * b;
WRITE_IO(c);
```

```
mov IO(0) -> RF(a0)     ; IO(0) is used to read data from outside
mov IO(0) -> RF(a1)     ; RF is a register file, to store data
mov RF(a1) -> mul(0)    ; mul(0) stores operand 1 of the multiplier
mov RF(a1) -> mul(1)    ; mul(1) stores operand 2 and triggers
mov mul(2) -> RF(a2)    ; mul(2) provides the multiplication result
mov RF(a0) -> add(0)    ; add(0) stores operand 1 of the adder
mov RF(a2) -> add(1)    ; b*b was stored to RF(a2) two lines before
mov add(2) -> IO(1)     ; IO(2) writes data to the outside
```




Computation example




```
a = READ_IO();
b = READ_IO();
c = a + b * b;
WRITE_IO(c);
```

```
mov IO(0) -> RF(a0)      ; IO(0) is used to read data from outside
mov IO(0) -> RF(a1)      ; RF is a register file, to store data
mov RF(a1) -> mul(0)     ; mul(0) stores operand 1 of the multiplier
mov RF(a1) -> mul(1)     ; mul(1) stores operand 2 and triggers
mov mul(2) -> RF(a2)     ; mul(2) provides the multiplication result
mov RF(a0) -> add(0)     ; add(0) stores operand 1 of the adder
mov RF(a2) -> add(1)     ; b*b was stored to RF(a2) two lines before
mov add(2) -> IO(1)      ; IO(2) writes data to the outside
```




Computation example

The program below is not optimal. What could be done better?

```
a = READ_IO();
b = READ_IO();
c = a + b * b;
WRITE_IO(c);
```

```
mov IO(0) -> RF(a0)     ; IO(0) is used to read data from outside
mov IO(0) -> RF(a1)     ; RF is a register file, to store data
mov RF(a1) -> mul(0)    ; mul(0) stores operand 1 of the multiplier
mov RF(a1) -> mul(1)    ; mul(1) stores operand 2 and triggers
mov mul(2) -> RF(a2)    ; mul(2) provides the multiplication result
mov RF(a0) -> add(0)    ; add(0) stores operand 1 of the adder
mov RF(a2) -> add(1)    ; b*b was stored to RF(a2) two lines before
mov add(2) -> IO(1)     ; IO(2) writes data to the outside
```




Computation example

The program below is not optimal.
What could be done better?

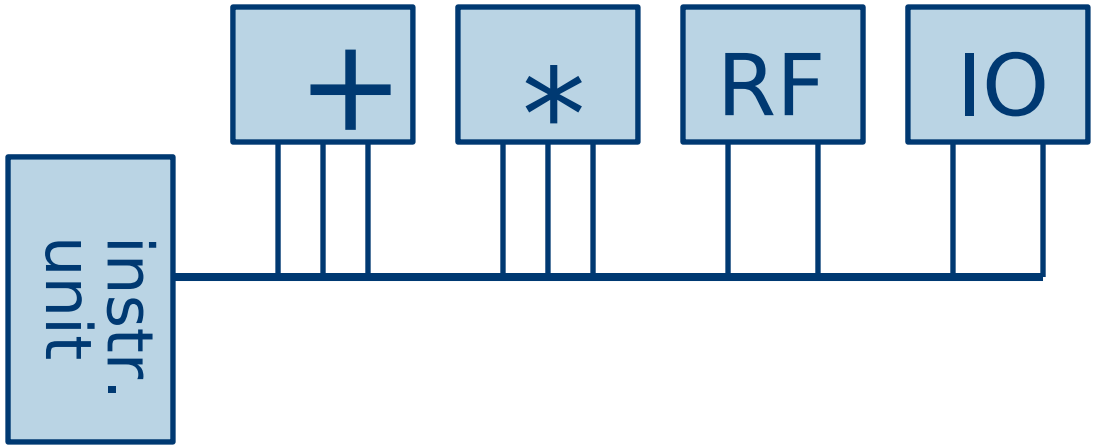
Circulating the data through RF is not necessary!

```
a = READ_IO();
b = READ_IO();
c = a + b * b;
WRITE_IO(c);
```

```
mov IO(0) -> RF(a0)     ; IO(0) is used to read data from outside
mov IO(0) -> RF(a1)     ; RF is a register file, to store data
mov RF(a1) -> mul(0)    ; mul(0) stores operand 1 of the multiplier
mov RF(a1) -> mul(1)    ; mul(1) stores operand 2 and triggers
mov mul(2) -> RF(a2)    ; mul(2) provides the multiplication result
mov RF(a0) -> add(0)    ; add(0) stores operand 1 of the adder
mov RF(a2) -> add(1)    ; b*b was stored to RF(a2) two lines before
mov add(2) -> IO(1)     ; IO(2) writes data to the outside
```

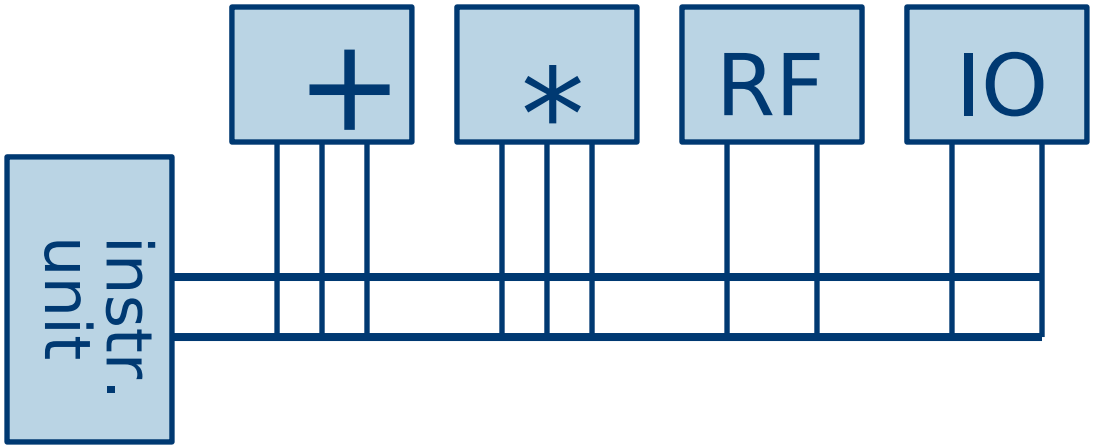



Multiple buses



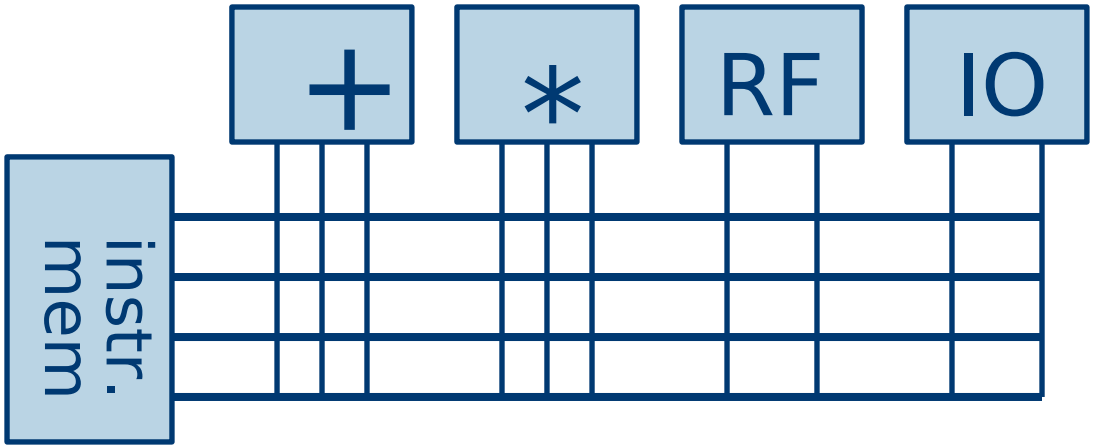
- This TTA processor has one bus. How would the functionality of the processor change if there would be a second bus?

Multiple buses



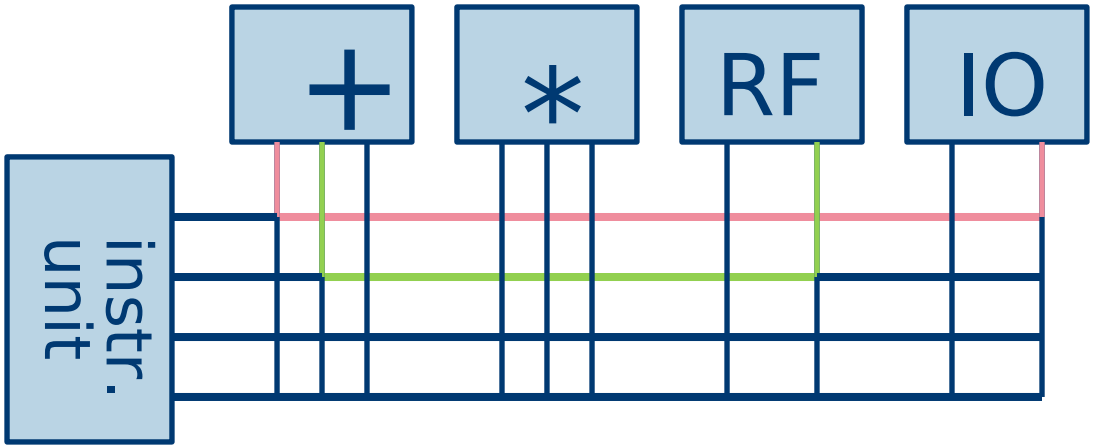
- Every additional bus adds a possibility for another parallel transfer

Multi-bus example



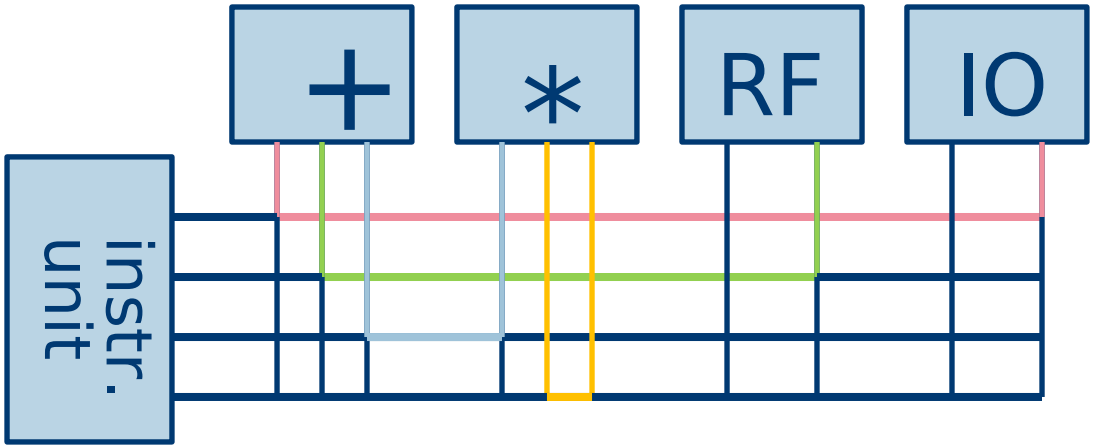
	Cycle 0	Cycle 1	Cycle 2	Cycle 3
Bus 0				
Bus 1				
Bus 2				
Bus 3				

Multi-bus example



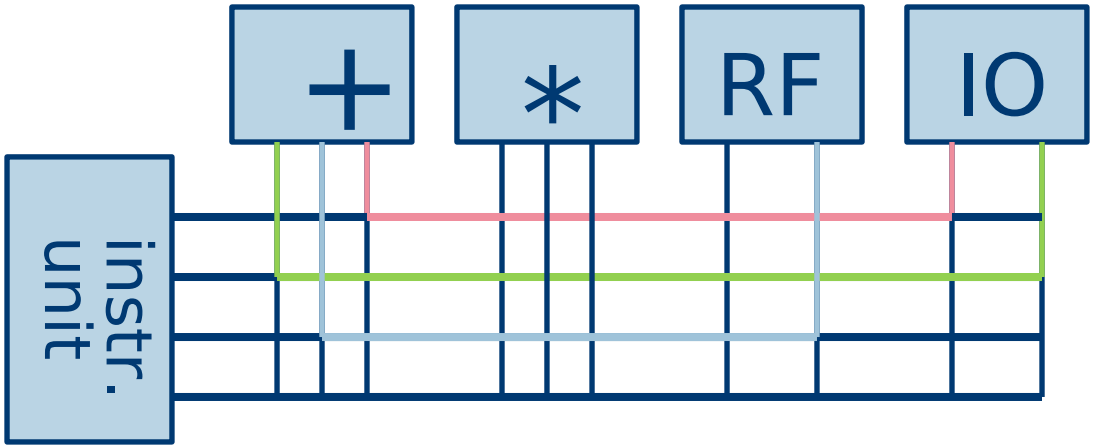
	Cycle 0	Cycle 1	Cycle 2	Cycle 3
Bus 0	mov IO(o1)→add(i1)			
Bus 1	mov RF(o1)→add(i2)			
Bus 2	...			
Bus 3	...			

Multi-bus example



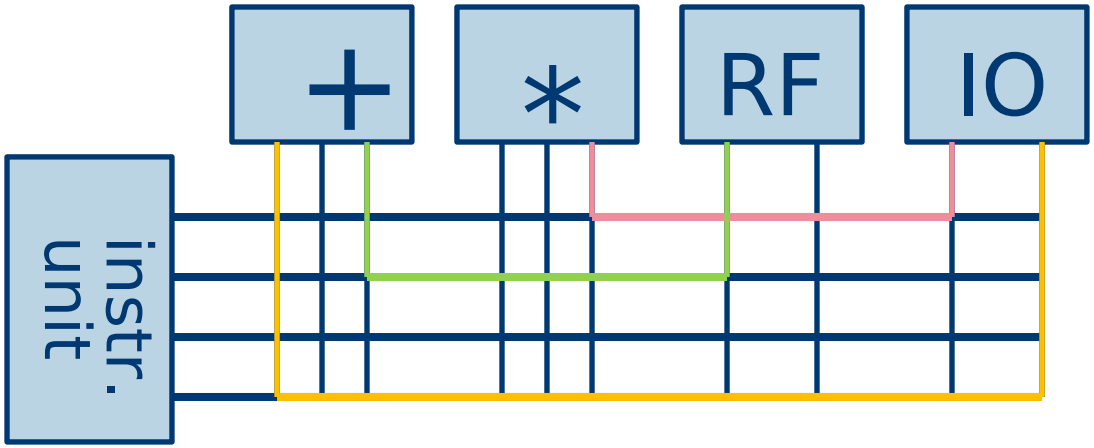
	Cycle 0	Cycle 1	Cycle 2	Cycle 3
Bus 0	mov IO(o1)→add(i1)	mov IO(o1)→add(i1)		
Bus 1	mov RF(o1)→add(i2)	mov RF(o1)→add(i2)		
Bus 2	…	mov add(o1)→mul(i1)		
Bus 3	…	mov mul(o1)→mul(i2)		

Multi-bus example



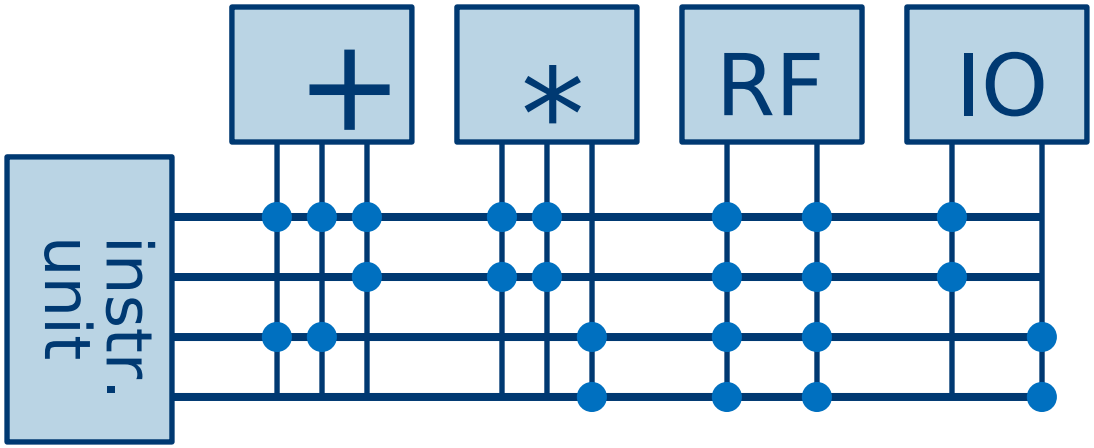
	Cycle 0	Cycle 1	Cycle 2	Cycle 3
Bus 0	mov IO(o1)→add(i1)	mov IO(o1)→add(i1)	mov add(o1)→IO(i1)	
Bus 1	mov RF(o1)→add(i2)	mov RF(o1)→add(i2)	mov IO(o1)→add(i1)	
Bus 2	…	mov add(o1)→mul(i1)	mov RF(o1)→add(i2)	
Bus 3	…	mov mul(o1)→mul(i2)	…	

Multi-bus example



	Cycle 0	Cycle 1	Cycle 2	Cycle 3
Bus 0	mov IO(o1)→add(i1)	mov IO(o1)→add(i1)	mov add(o1)→IO(i1)	mov mul(o1)→IO(i1)
Bus 1	mov RF(o1)→add(i2)	mov RF(o1)→add(i2)	mov IO(o1)→add(i1)	mov add(o1)→RF(i1)
Bus 2	…	mov add(o1)→mul(i1)	mov RF(o1)→add(i2)	…
Bus 3	…	mov mul(o1)→mul(i2)	…	mov IO(o1)→add(i1)

Multiple buses

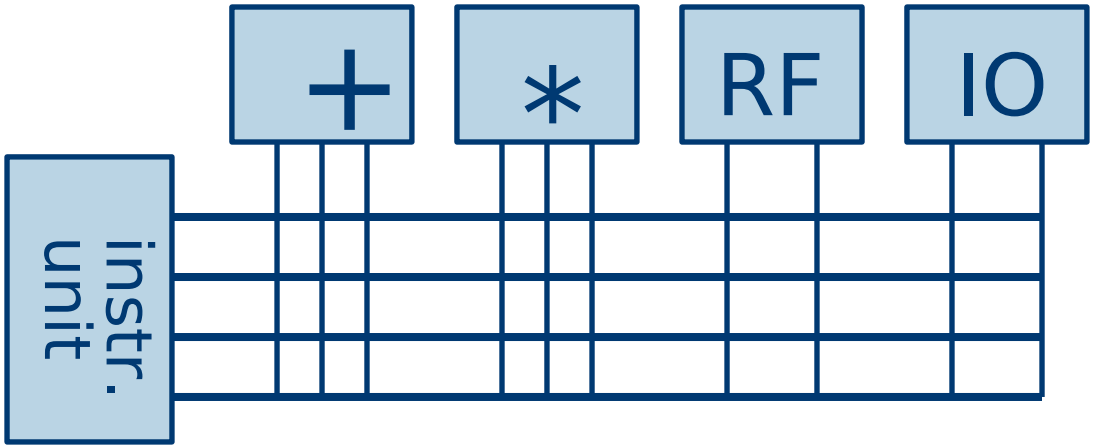


- Going into detail, all sockets are actually *not* connected to every bus.
- Less connections means lower power consumption.

TTA instructions

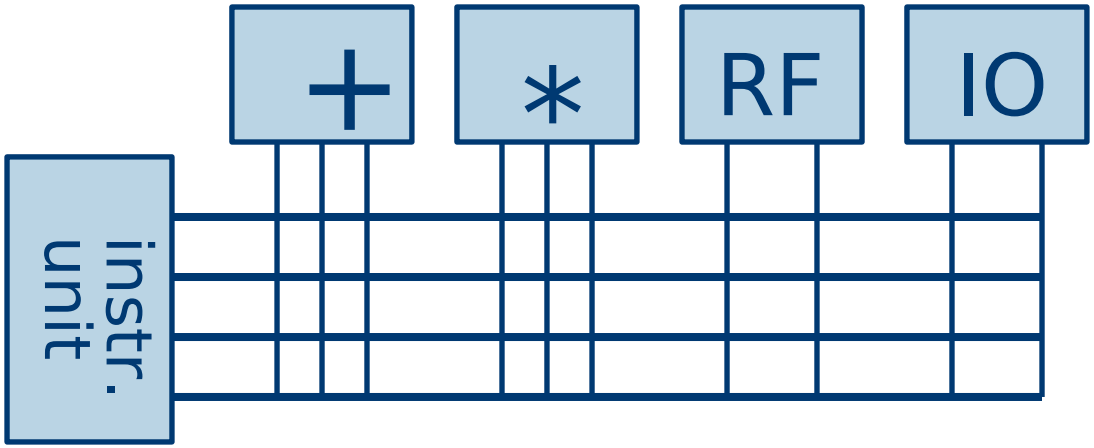


TTA instructions



- But how do the TTA instructions look like in binary format?

TTA instructions



0000110100011 ... 00000011101010101000

168 bits
for one instruction
→ 42 bits for each bus

TTA instructions

Each bus needs a 42 bit instruction each clock cycle. Where do the 42 bits come from?

-

-

-

-

-

How wide is an 8-bus TTA instruction?



TTA instructions

Each bus needs a 42 bit instruction each clock cycle. Where do the 42 bits come from?

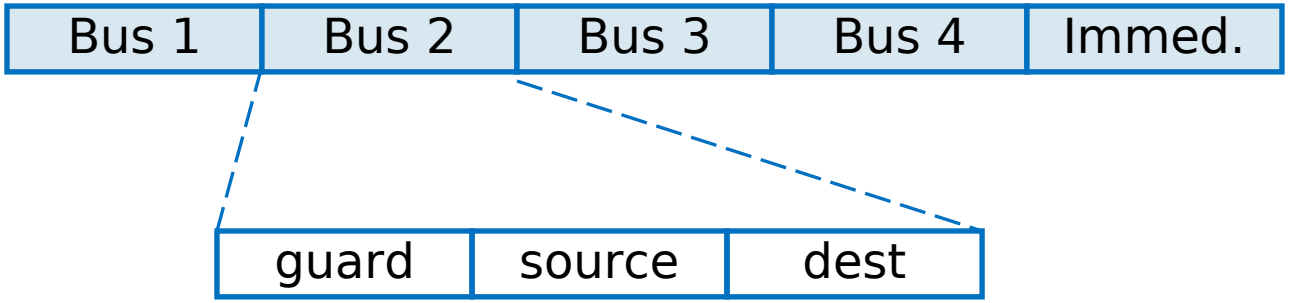
- source port
- destination port
- opcode
- guard bits
- immediate values

How wide is an 8-bus TTA instruction?
336b



TTA instructions

Instruction word



TTA instructions

- Very long instruction words (like 168 or 336 bits) require a lot of program memory space if the program is long
- To make the problem less severe, *instruction compression* techniques exist
- Instruction compression is based on a dictionary: compressed instructions are just index number that point to the full instruction in the dictionary



Performance optimization



Performance optimization

The SW/HW designer of TTA processors must know the central issues about performance optimization

- How the algorithm works
- What resources the algorithm needs
- Understand how the C compiler works



Performance optimization

- The strength of TTA processors is that they can directly route data from one place to another, without obligatory register/memory stores
- Memory accesses are slow

→ the program should only access data memory when really necessary



Performance optimization

```
unsigned char fp_fir(unsigned char input, unsigned char *H)
{
        unsigned short j;
        unsigned short accu = 0;

        for(j = 0; j < FILTLEN-1; j++)
                delayline[j] = delayline[j+1];

        delayline[FILTLEN-1] = input;

        for(j = 0; j < FILTLEN; j++)
                accu += delayline[3-j] * H[j];

        return accu >> 8;
}
```

- The TTA processor for this code should have so much register space that memory accesses are not needed for this loop



Performance optimization

```
unsigned char fp_fir(unsigned char input, unsigned char *H)
{
        unsigned short j;
        unsigned short accu = 0;

        for(j = 0; j < FILTLEN-1; j++)
                delayline[j] = delayline[j+1];

        delayline[FILTLEN-1] = input;

        for(j = 0; j < FILTLEN; j++)
                accu += delayline[3-j] * H[j];

        return accu >> 8;
}
```

- By examining the assembly code (output of the C compiler), one can see if the loop has accesses the load-store unit (LSU).
- If it does, memory is accessed



Performance optimization

Bus 1	Bus 2	Bus 3	Bus 4
326 -> RF_2.1	RF_1.0 -> add.in1t.add	4 -> add.in2	0 -> io_in.in.stream_in
4 -> add.in2	io_in.out -> lsu.in2	add.out1 -> lsu.in1t.stw	325 -> RF_1.3
255 -> and_ior.in2	RF_1.0 -> add.in1t.add	RF_2.1 -> lsu.in1t.ldqu	327 -> RF_1.2
4 -> add.in2	37 -> mul_2.in2	109 -> mul_1.in2	add.out1 -> RF_1.1
1 -> logic.in2	0 -> alu_comp.in2	0 -> io_in.in.stream_in_status	RF_1.0 -> add.in1t.add
...	RF_1.3 -> lsu.in1t.stq	lsu.out1 -> RF_2.2	lsu.out1 -> lsu.in2

- By examining the assembly code (output of the C compiler), one can see if the loop has accesses the load-store unit (LSU).
- If it does, memory is accessed



Performance optimization

- The functionality of a signal processor must be balanced for high efficiency (low gate count, high throughput)
- FIR example: You start with a processor that has 1 multiplier and 1 adder. You want to make the processor 3 times faster.

→ if you make the processor have 3 multipliers, you probably also need 3 adders



Performance optimization

- Profiling tools are used to see if the processor is balanced
- Things to look for:
  - if there is a FU that is used much more often than others, it probably is a bottleneck
  - if there is a FU that has (almost) no accesses, it can be removed to save on gate count

